

Транзакции. TSL.

Лекция 7

Целостность данных

Целостность данных – это корректность данных, соответствие некоторой структуре и логике, соблюдение инвариантов

- Физическая (защита данных при отказе БД, разрыве сети, повреждении диска и пр.)
- Логическая
 - Сущностная (наличие уникального непустого (non null) ключа)
 - Ссылочная (поддержка внешних ключей; проверка корректности ссылок)
 - Доменная (поддержка типов данных; unique, not null, enumerate ограничений; ограничения на диапазон данных)
 - Пользовательская (в группе должен быть хотя бы один человек, на дежурстве должно быть не меньше 2х врачей)

Транзакции

“An SQL-transaction is a sequence of executions of SQL-statements that is atomic with respect to recovery” ISO/IEC 9075(ISO/IEC 9075:1999)

Это группа последовательных операций над данными, объединенных в логическую единицу работы (unit of work).

Транзакции **не должны** нарушать целостность данных. Это достигается благодаря следованию ACID.

- A (Atomicity) атомарность
- C (Consistency) согласованность
- I (Isolation) изоляция
- D (Durability) устойчивость, сохраняемость

Управление транзакциями

```
BEGIN; -- начало транзакции
```

```
-- блок транзакции тут
```

```
COMMIT; -- фиксация изменений
```

```
BEGIN; -- начало транзакции
```

```
-- блок транзакции тут
```

```
ROLLBACK; -- откат изменений
```

Вместо **BEGIN** можно использовать **START TRANSACTION**;

Пример

```
BEGIN;
```

```
-- Шаг 1: Создаем запись о выдаче
```

```
INSERT INTO book_loans (book_id, reader_id, loan_date, due_date, status)  
VALUES (1, 5, CURRENT_DATE, CURRENT_DATE + 14, 'active');
```

```
-- Шаг 2: Уменьшаем количество доступных книг
```

```
UPDATE books  
SET available_quantity = available_quantity - 1  
WHERE book_id = 1;
```

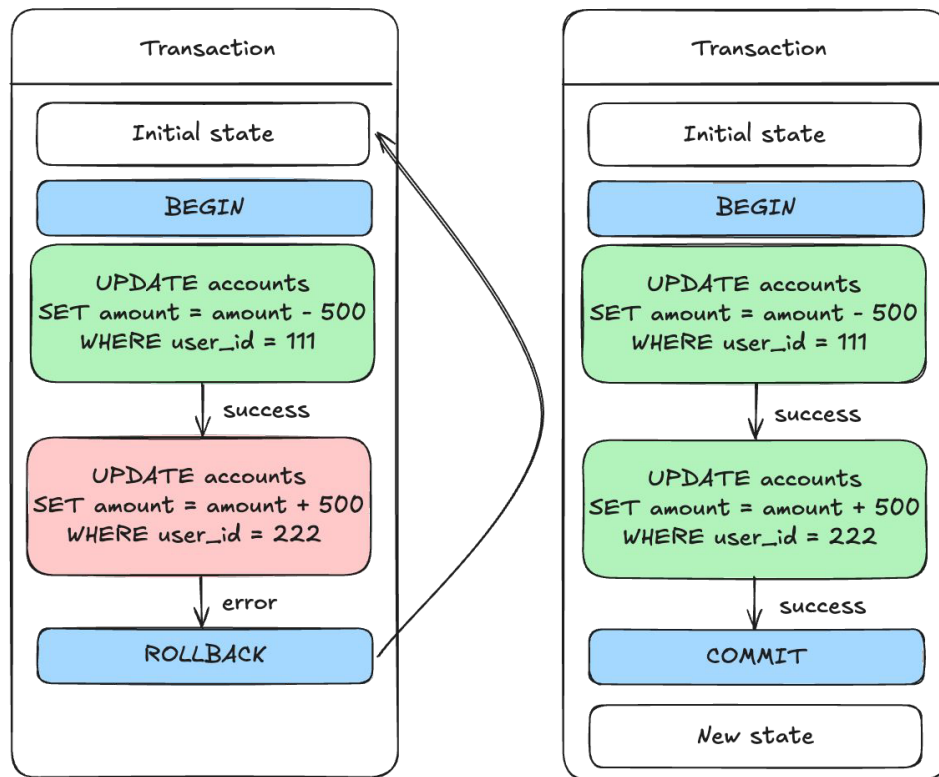
```
-- Проверяем (внутри транзакции видим изменения)
```

```
SELECT available_quantity FROM books WHERE book_id = 1;
```

```
COMMIT;
```

Атомарность (A)

“Все или ничего!” – означает невозможность разбиения на более мелкие части



Согласованность (C)

При фиксации изменения должны выполняться все ограничения целостности

Примеры ограничений: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT

Однако некоторые ограничения определяются на уровне приложения.

Примеры: нет пересекающихся по времени бронирований столиков в ресторане, сумма оплаты в заказе совпадает с суммой всех товаров в корзине.

Можем ли вставить строки?

```
CREATE TABLE wives (id int not null, husband_id int NOT null, PRIMARY KEY(id));  
CREATE TABLE husbands (id int not null, wife_id int NOT null, PRIMARY KEY(id));  
  
ALTER TABLE wives ADD CONSTRAINT h_k FOREIGN KEY(husband_id) REFERENCES husbands;  
ALTER TABLE husbands ADD CONSTRAINT w_k FOREIGN KEY(wife_id) REFERENCES wives;
```

Пытаемся вставить строки:

```
BEGIN;  
  
INSERT INTO wives (id, husband_id) VALUES (1, 1);  
  
INSERT INTO husbands (id, wife_id) VALUES (1, 1);  
  
COMMIT;
```


Проблема: ограничения блокируют операции

```
CREATE TABLE wives (id int not null, husband_id int NOT null, PRIMARY KEY(id));
```

```
CREATE TABLE husbands (id int not null, wife_id int NOT null, PRIMARY KEY(id));
```

```
ALTER TABLE wives ADD CONSTRAINT h_k FOREIGN KEY(husband_id) REFERENCES husbands;
```

```
ALTER TABLE husbands ADD CONSTRAINT w_k FOREIGN KEY(wife_id) REFERENCES wives;
```

Пытаемся вставить строки:

```
BEGIN;
```

```
INSERT INTO wives (id, husband_id) VALUES (1, 1);
```

```
INSERT INTO husbands (id, wife_id) VALUES (1, 1);
```

```
COMMIT;
```

```
[23503] ERROR: insert or update on table "wives" violates foreign key constraint "h_k"  
Detail: Key (husband_id)=(1) is not present in table "husbands".
```

Отложенные (deferrable) ограничения

```
ALTER TABLE wives ADD CONSTRAINT h_k FOREIGN KEY(husband_id) REFERENCES husbands;
```

По-умолчанию ограничения NOT DEFERRABLE – проверка происходит сразу.

Но можно указать DEFERRABLE ограничения:

- DEFERRABLE INITIALLY DEFERRED – проверка отложена на конец транзакции
- DEFERRABLE INITIALLY IMMEDIATE – можно отложить проверку на конец транзакции, но по-умолчанию проверяем сразу

Решение проблемы:

```
ALTER TABLE wives ALTER CONSTRAINT h_k DEFERRABLE INITIALLY DEFERRED;
```

```
ALTER TABLE husbands ALTER CONSTRAINT w_k DEFERRABLE INITIALLY DEFERRED;
```

SET CONSTRAINTS

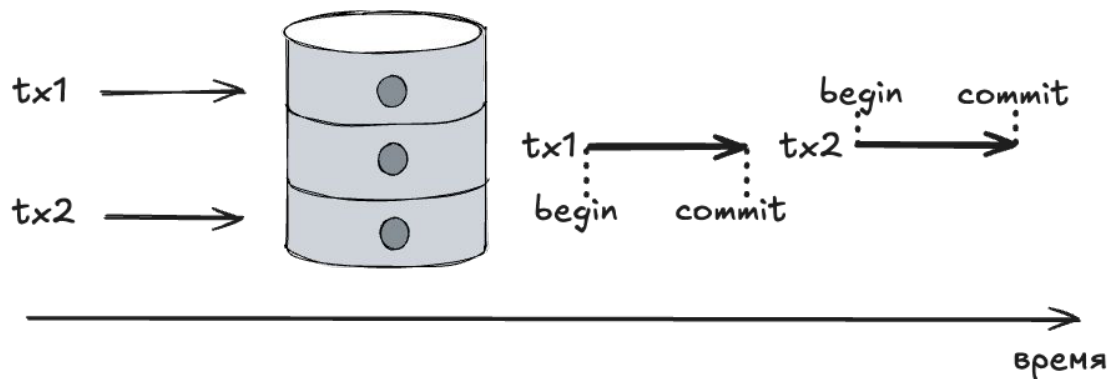
```
ALTER TABLE wives ALTER CONSTRAINT h_k DEFERRABLE INITIALLY IMMEDIATE;  
ALTER TABLE husbands ALTER CONSTRAINT w_k DEFERRABLE INITIALLY IMMEDIATE;  
  
BEGIN;  
  
SET CONSTRAINTS h_k, w_k DEFERRED; -- откладываем проверку  
  
INSERT into wives (id, husband_id) VALUES (2, 2);  
  
INSERT INTO husbands (id, wife_id) VALUES (2, 2);  
  
COMMIT;  
  
SET CONSTRAINTS ... IMMEDIATE — включает немедленную проверку
```

Ограничения NOT NULL и CHECK всегда проверяются немедленно

Изоляция (I)

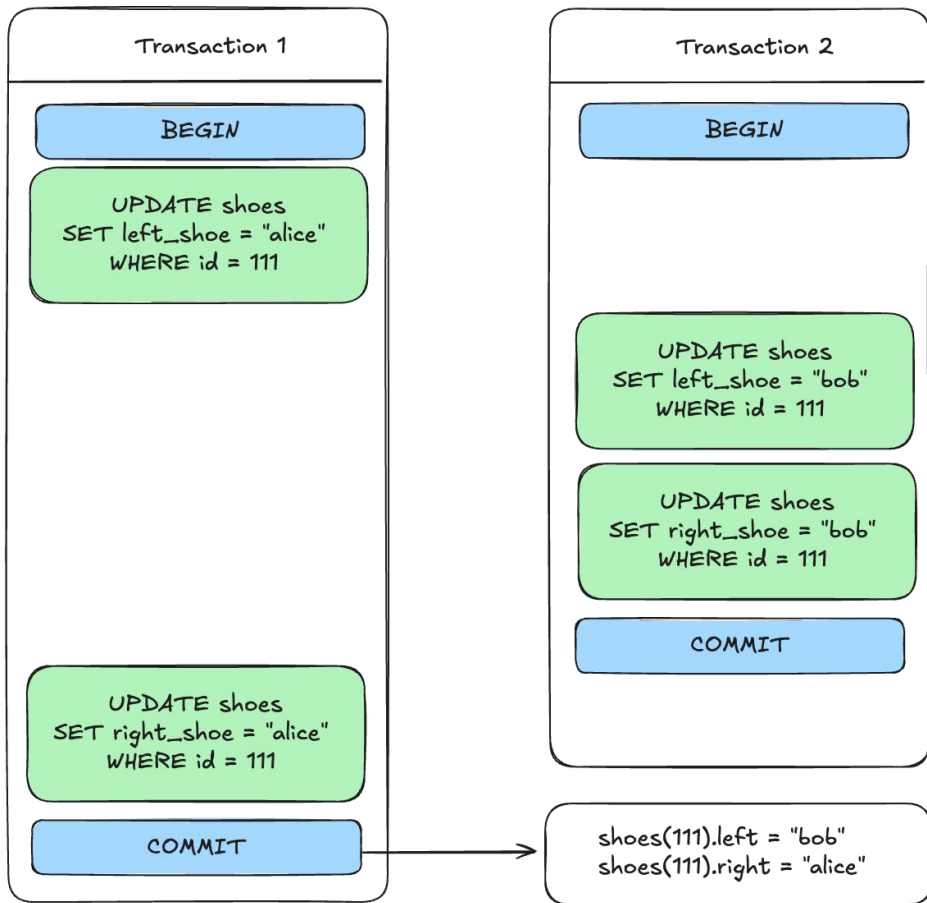
Конкурентно выполняемые транзакции не могут помешать друг другу.

Изоляция тождественна **сериализуемости**: транзакции выполняются так, будто каждая из них единственная во всей СУБД в конкретный момент времени, а результат их выполнения такой же, как если бы транзакции выполнялись последовательно.



На практике вместо сериализуемости СУБД идут на компромиссы, допуская нарушение согласованности, вводя различные **уровни изоляции**.

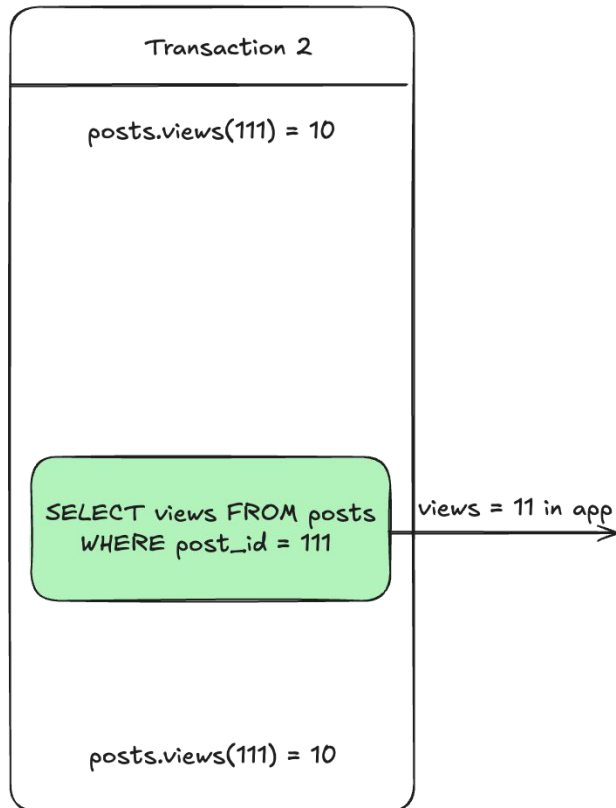
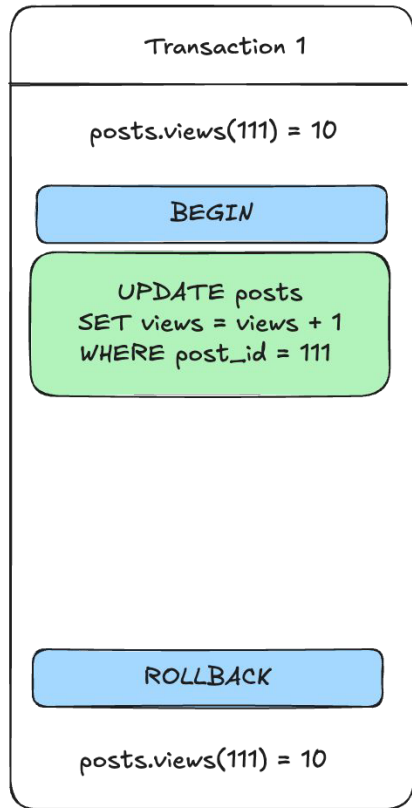
“Грязная” запись



Решение: транзакция 1 при попытке обновить запись устанавливает на нее блокировку, транзакция 2 ждет, пока блокировка освободится.

На практике такое нарушение согласованности разрешается при любых уровнях изоляции в любых СУБД, т.е. фактически **не происходит никогда**.

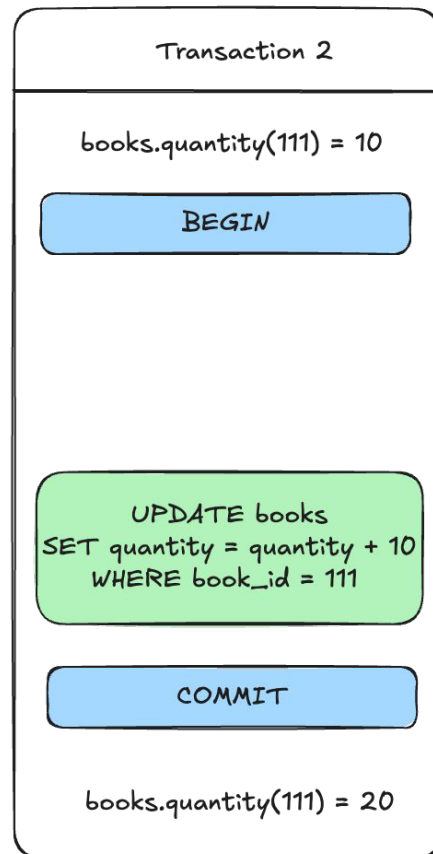
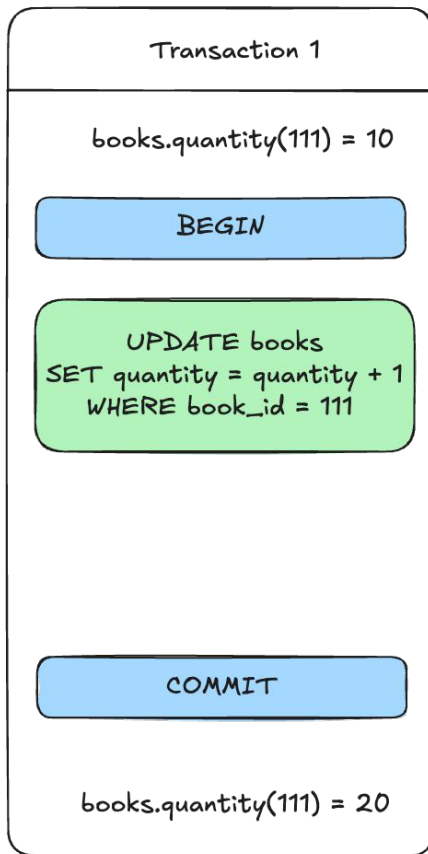
“Грязное” чтение



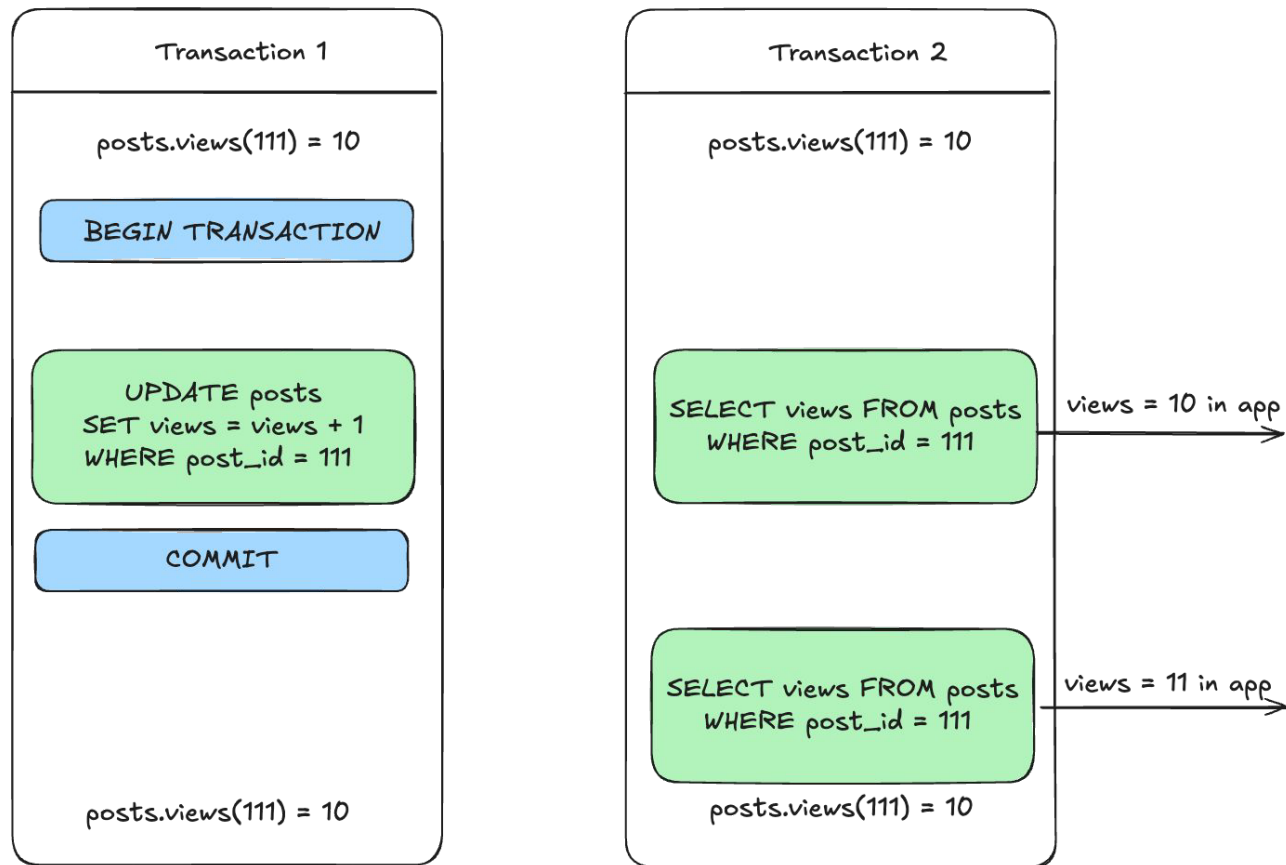
Решение: multiversion concurrency control (MVCC). СУБД хранит зафиксированную версию строки и перезаписанную, но не зафиксированную.

Каждая транзакция работает со своей версией строк. Снимок данных создается перед выполнением каждого запроса.

Потерянное обновление

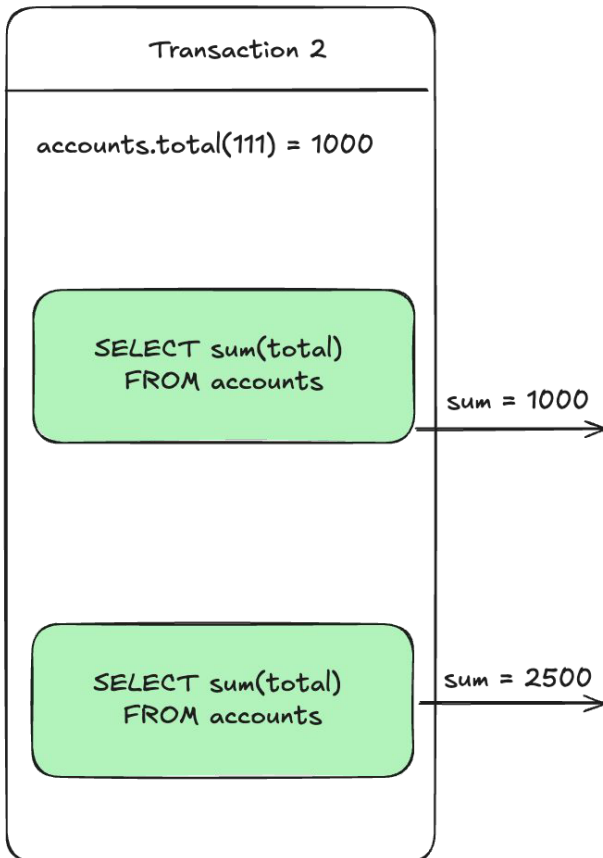
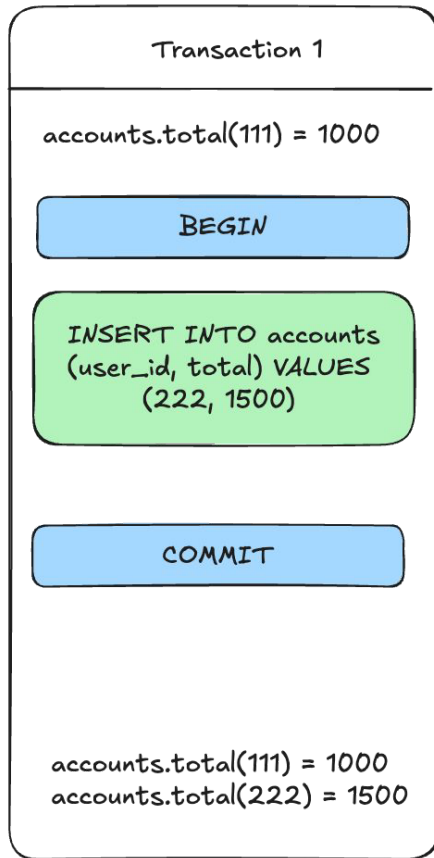


Неповторяющееся чтение



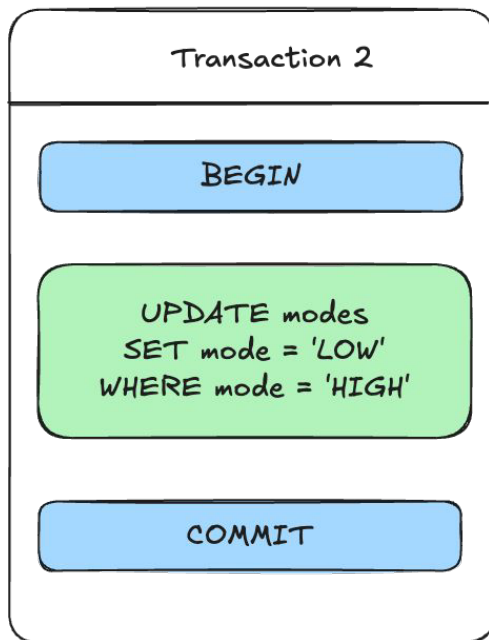
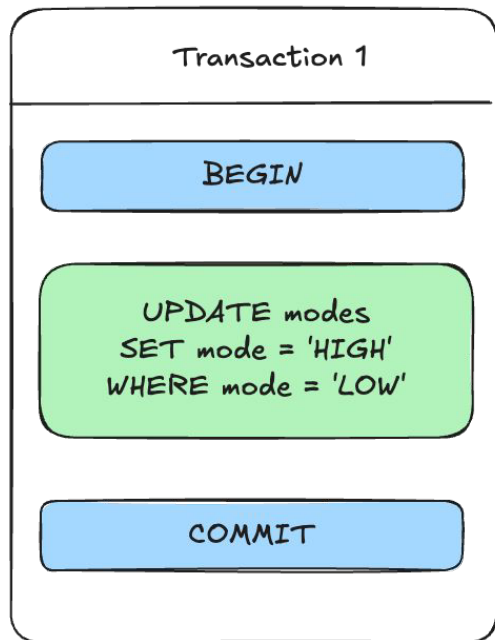
Решение: multiversion concurrency control (MVCC). Снимок данных создается один раз при старте транзакции.

Чтение “фантомов”



Аномалия сериализации

```
CREATE TABLE modes (num integer, mode text);  
INSERT INTO modes VALUES ( 1, 'LOW' ), ( 2, 'HIGH' );
```



Было:

num	mode
1	LOW
2	HIGH

Стало:

num	mode
1	HIGH
2	LOW

Но какому порядку
выполнения транзакций
соответствует результат?



Уровни изоляции по стандарту SQL

	“грязная” запись	“грязное” чтение	потерянное обновление	неповторяю щееся чтение	чтение “фантомов”	аномалии сериализац ии
read uncommitted	+	-	-	-	-	-
read committed	+	+	-	-	-	-
repeatable read	+	+	+	+	-	-
serializable	+	+	+	+	+	+

+ – на этом уровне аномалия невозможна

Уровни изоляции Postgres

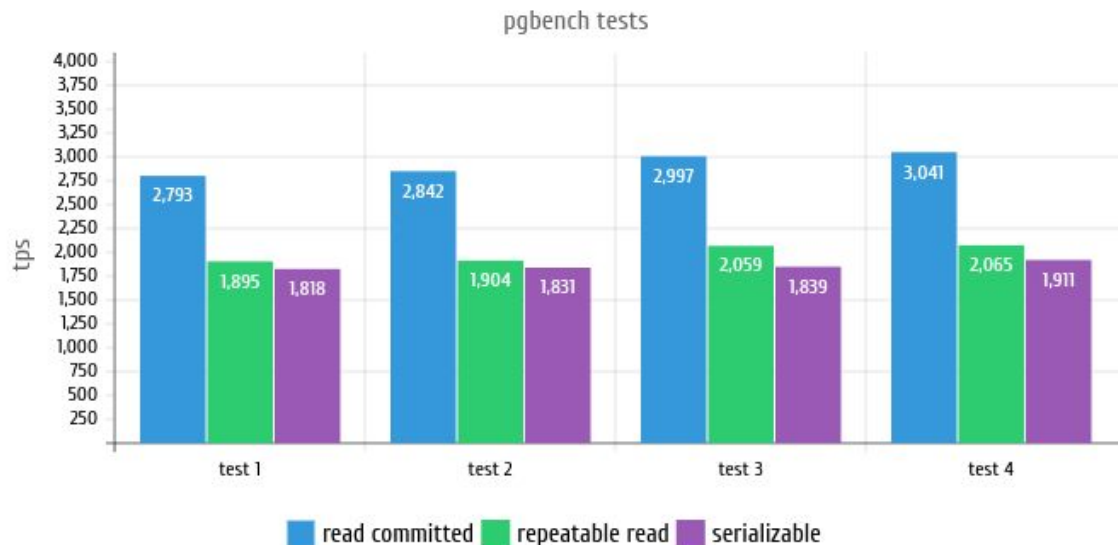
	“грязная” запись	“грязное” чтение	потерянное обновление	неповторяю щееся чтение	чтение “фантомов”	аномалии сериализац ии
read uncommitted	+	+	+	-	-	-
read committed	+	+	+	-	-	-
repeatable read	+	+	+	+	+	-
serializable	+	+	+	+	+	+

По-умолчанию используется read committed

Производительность

Чем выше уровень изоляции, тем ниже пропускная способность Postgres

Postgresql Read committed, repeatable read, serializable performance



Источник: <https://habr.com/ru/articles/857486> (Уровни изоляции транзакций в PostgreSQL, MySQL, MS SQL, Oracle с примерами на Go)

Смена уровня изоляции

На уровне транзакции:

```
BEGIN;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

или просто

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

На уровне сессии:

```
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

На уровне кластера: задать default_transaction_isolation в postgresql.conf

Посмотреть текущий уровень:

```
SHOW default_transaction_isolation;
```

MVCC в Postgres

Девиз: **чтение не блокирует запись, а запись чтение**

Каждая новая транзакция получает возрастающий XID

Любая строка в СУБД:

xmin	xmax	остальные поля...
------	------	-------------------

xmin – транзакция, создавшая запись, xmax – удалившая

Строка видна в текущей транзакции XID_curr, если выполняются оба условия

1. xmin < XID_curr и xmin – зафиксирована
2. xmax = 0 (строка не удалена) или xmax не зафиксирована

Сериализуемость

Способы достижения

- Последовательное выполнение транзакций – нереализуемо на практике
- Сериализуемая изоляция снимков состояний (Serializable Snapshot Isolation, SSI) – техническая реализация уровня изоляции Serializable в Postgres. <https://arxiv.org/abs/1208.4179>. Требуется смены уровня изоляции.
- Двухфазная блокировка – не требует смены уровня изоляции, контролируется клиентом, но возможны dead locks и понижение производительности

Блокировки уровня строк

- FOR UPDATE (`SELECT * FROM test FOR UPDATE`) -> может удерживать только одна транзакция, блокирует все операции над строкой, а также FOR SHARE
- FOR SHARE (`SELECT * FROM test FOR SHARE`) -> может удерживаться несколькими транзакциями, блокирует FOR UPDATE

Более слабые блокировки, включающие операции с foreign keys

- FOR NO KEY UPDATE -> аналогично FOR UPDATE, но не блокирует FOR KEY SHARE
- FOR KEY SHARE -> аналогично FOR SHARE, но не блокирует FOR NO KEY UPDATE, допускает операции, если они не меняют значения ключей

<https://postgrespro.ru/docs/shardman/18/explicit-locking#LOCKING-ROWS>

Блокировка FOR UPDATE (пример)

```
BEGIN;  
    -- Выбираем и блокируем строку  
    SELECT book_id, title, available_quantity FROM books  
    WHERE book_id = 1 FOR UPDATE;  
  
    -- 🔒 Строка заблокирована  
    -- Другие сессии НЕ могут:  
    -- - UPDATE этой строки (будут ждать)  
    -- - DELETE этой строки (будут ждать)  
    -- - SELECT ... FOR UPDATE (будут ждать)  
  
    -- Обновляем  
    UPDATE books  
    SET available_quantity = available_quantity - 1  
    WHERE book_id = 1;  
COMMIT;  -- 🔓 Блокировка снята
```

Блокировка FOR SHARE (пример)

BEGIN;

-- Выбираем с разделяемой блокировкой

SELECT book_id, title, available_quantity

FROM books

WHERE book_id = 2

FOR SHARE;

--   Строка заблокирована для чтения

-- Другие сессии МОГУТ:

-- - SELECT (обычный) 

-- - SELECT ... FOR SHARE  (тоже могут читать)

-- Другие сессии НЕ могут:

-- - UPDATE этой строки (будут ждать)

-- - DELETE этой строки (будут ждать)

-- - SELECT ... FOR UPDATE (будут ждать)

COMMIT;

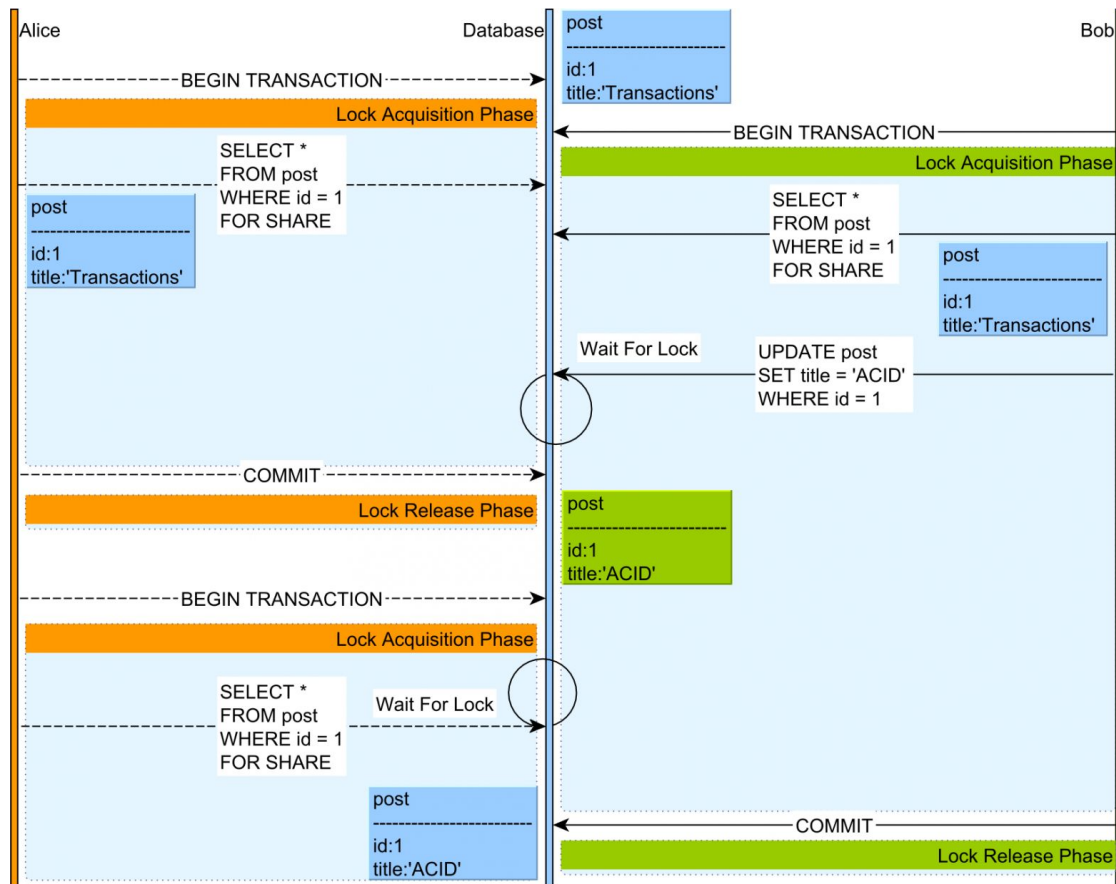
Двухфазная блокировка (2-phase lock, 2PC)

При чтении устанавливается shared блокировка. Перед записью устанавливается exclusive блокировка. Но exclusive не может быть установлена, если уже установлена shared другой транзакции, и наоборот.

Две фазы:

- Фаза расширения: транзакция только пытается установить блокировки (либо повысить уровень с shared до exclusive) во время непосредственно выполнения
- Фаза сжатия: транзакция заканчивает выполнение (успешно или неуспешно) и освобождает все блокировки

Двухфазная блокировка (пример)



Виды блокировок

- Уровня строк: FOR SHARE, FOR UPDATE
- Уровня таблиц: ACCESS EXCLUSIVE и другие
 - `LOCK TABLE books IN ACCESS EXCLUSIVE MODE;` *-- блокирует все операции с таблицей в других транзакциях*
- Рекомендательная блокировка (advisory lock) – блокировка, смысл которой задает приложение
 - `SELECT pg_advisory_lock(id) FROM foo WHERE id = 12345;`

Сохраняемость (D)

Обязательство СУБД не терять данные после подтверждения фиксации транзакции, даже в случае аппаратного или фатального сбоя.

Обычно достигается за счет использования WAL (write-ahead log) – append-only журнала на диске, где фиксируются все операции, прежде, чем СУБД обновит данные в основном хранилище.

WAL
SET key1 value1
SET key2 value2
DEL key1
SET key3 value3
SET key1 value1

Storage
key1 = value1
key2 = value2
key3 = value3

Точки сохранения

```
BEGIN;
```

```
-- Операция 1: Выдаем первую книгу
```

```
INSERT INTO book_loans (book_id, reader_id, loan_date, due_date, status)
VALUES (1, 5, CURRENT_DATE, CURRENT_DATE + 14, 'active');
```

```
UPDATE books SET available_quantity = available_quantity - 1
WHERE book_id = 1;
```

```
SAVEPOINT after_first_book; -- 📌 Точка сохранения
```

```
-- Операция 2: Пытаемся выдать вторую книгу
```

```
INSERT INTO book_loans (book_id, reader_id, loan_date, due_date, status)
VALUES (2, 5, CURRENT_DATE, CURRENT_DATE + 14, 'active');
```

```
-- Проверяем: книги нет в наличии!
```

```
SELECT available_quantity FROM books WHERE book_id = 2; -- 0!
```

```
ROLLBACK TO after_first_book; -- ⏮ Откатываемся к точке
```

```
-- Операция 2 отменена, но операция 1 сохранена!
```

```
COMMIT; -- ✅ Сохраняем: первая книга выдана, вторая - нет
```